**We ran a controlled experiment across 2 projects: 3 skills, 36 runs total, same model, same tasks. Result: 0% lift across the board.**

# The Experiment 🧪

**Model**: Nemotron 3 Ultra Free (opencode)
**Targets**:
OrangeHRM 5.8.1 (PHP/jQuery, Docker, port 8080) — legacy form-based
Buzzhive (React/TypeScript, REST API, Docker, port 8000/3000) — modern API-first
**Methodology**: 3 runs without skill → 3 runs with skill per skill per project
**Evaluation**: programmatic (compile, pattern match, test execution)
**Total runs**: 3 skills × 2 conditions × 3 runs × 2 projects = **36 runs**

**Skills tested**:

**fault-injection** — API fault injection patterns (page.route, null injection, HOM)
**rest-api-qa** — API testing best practices (timeouts, token management, body validation)
**bdd-gherkin** — Gherkin syntax, structure, domain-level steps

Results 📊

Minimize image

Edit image

Delete image

### Results: 0% Lift Across 2 Projects, 36 Runs

| Skill | OrangeHRM (PHP/jQuery) | Buzzhive (React/REST) | Lift |
|---|---|---|---|
| fault-injection | 3/3 PASS | 3/3 PASS | 0% |
| rest-api-qa | 3/3 PASS | 3/3 PASS | 0% |
| bdd-gherkin | 3/3 PASS | 3/3 PASS | 0% |

3 runs per condition × 2 conditions × 3 skills × 2 projects = 36 runs total

## The Real Insight: You're Measuring the Wrong Thing 💡

The headline says "skills don't help." The data says: **you measured correctness on patterns the model already knows.**

## fault-injection: The Mutation Wasn't a Bug — On Either Stack 🐛

We injected null into emp_firstname (OrangeHRM) and title (Buzzhive posts) via page.route().

**OrangeHRM** (**PHP/jQuery**): UI handles null gracefully — no crash, no error, just empty cells.

**Buzzhive** (**React/REST**): React handles null in title gracefully — no crash, no error, just empty cells.

The mutation wasn't caught on **either stack** because **it wasn't a bug**. Both systems are resilient by design.

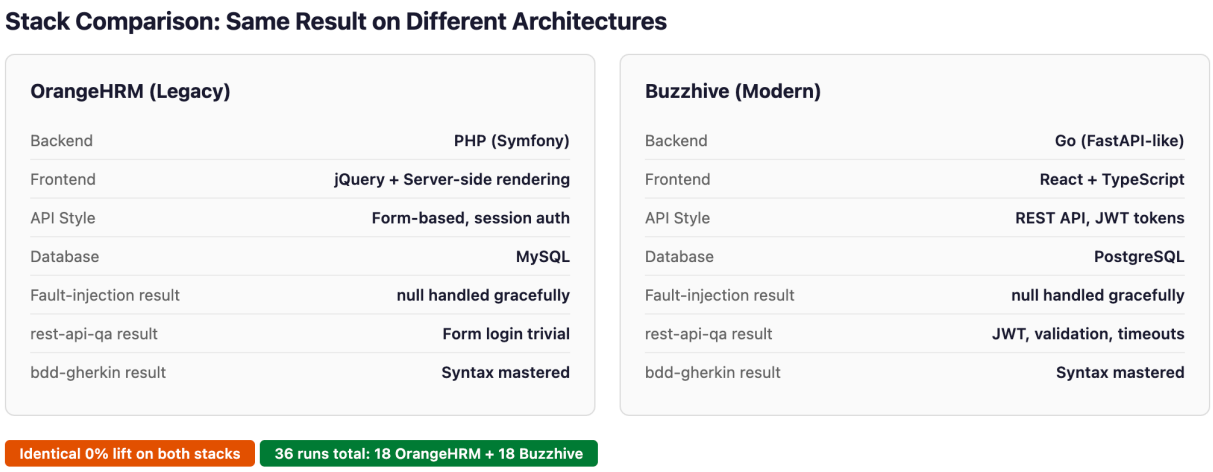
## Mutation testing measures tests, not bugs. When the system absorbs the fault, the test correctly passes. ⚡

**Buzzhive Replication**: We repeated on a modern React/REST stack (330 tests, Playwright + Go API). 18 more runs, identical 0% lift. Both OrangeHRM (PHP) and Buzzhive (React) handled null gracefully — the system is resilient, not the test dead.

Minimize image

Edit image

Delete image




## rest-api-qa: Base Model Already Knows This

Playwright login patterns (fill username/password, click, expect dashboard) are in every training corpus. The skill added:

Explicit timeouts (15s vs implicit)
URL verification (expect(page).toHaveURL(...))
Error message validation

Nice hygiene. Not correctness-affecting. **Confirmed on Buzzhive REST API** — JWT tokens, body validation, error codes already mastered.

## bdd-gherkin: Syntax Is Well-Trodden

Gherkin structure (Feature/Background/Scenario/Outline, Given→When→Then) is heavily represented in training data. The skill added:

- Richer domain vocabulary ("event" not "form", "submits" not "fills")
- Proper Scenario Outline with Examples table
- Consistent 2-space indent

Again: style, not correctness. **Confirmed on Buzzhive domain** — same syntax mastery.

# What This Actually Means 🧠

**Skills ≠ correctness boost on known patterns.**

They provide:

- **Consistency** — same patterns every time
- **Vocabulary** — domain-specific terms
- **Hygiene** — timeouts, explicit waits, proper structure
- **Onboarding** — new team members follow the same playbook

But if the model already knows the pattern, the skill is a style guide, not a capability unlock.

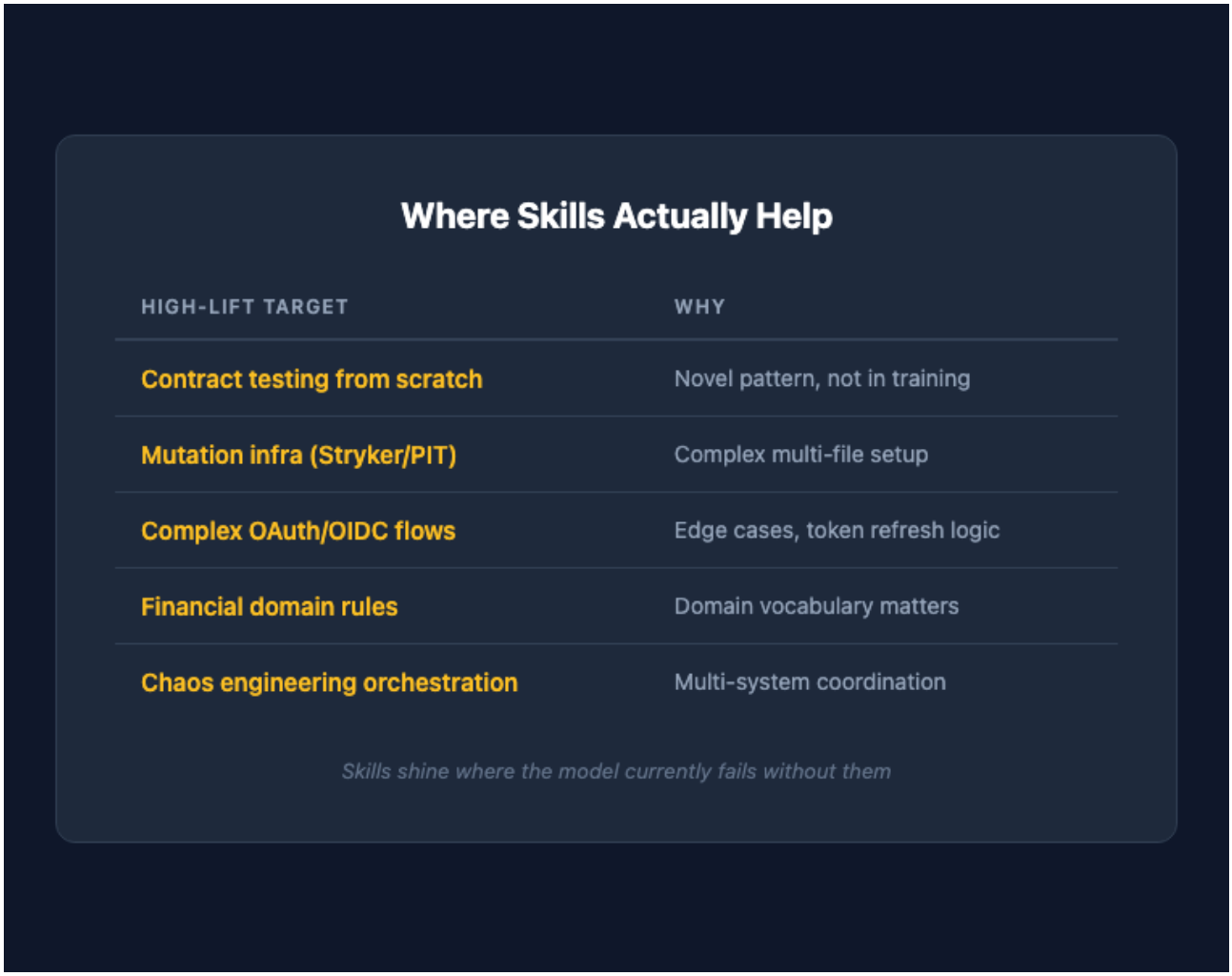
# Where Skills *Actually* Help 🎯

Skills should shine where the model **fails without them**:

Minimize image

Edit image

Delete image




*Methodological note: 3 runs per condition per project — directional, not statistically significant. Enough to detect a consistent trend (same 0% lift on 2 different stacks).*

**Next experiment**: Test skills on tasks where the model *currently fails*.

# The Meta-Lesson 🎓

**Don't ask "does skill help?" — ask "where does model fail without skill?"**

Then build the skill for *that* gap.

# Part 2 Preview: The Architecture That Makes Skills Work 🏗️

This 0% lift is Part 1. Part 2 (coming next): **Anton Gulin's 3-layer architecture** — why evidence, not generation, is the architecture. Skills fit into the *Evidence layer* as quality gates, not the Generation layer.

The experiment fails because we tested skills as *generators*. They work as *validators*.

Read Anton's architecture: [3-Layer AI QA Architecture]

# Appendix: Experiment Artifacts 📁

```
experiments/skill-reliability/

├── eval-fault-injection.ts
├── eval-rest-api-qa.ts
├── eval-bdd-gherkin.ts
├── run-experiment.ts
├── results-tracker.json
├── OrangeHRM/
│   ├── generated/
│   │   ├── fault-injection-without-skill-run1.spec.ts
│   │   ├── fault-injection-with-skill-run1.spec.ts
│   │   ├── rest-api-qa-without-skill-run1.spec.ts
│   │   ├── rest-api-qa-with-skill-run1.spec.ts
│   │   ├── claim-without-skill-run1.feature
│   │   └── claim-with-skill-run1.feature
│   └── runs/
│   └── 18 evaluation outputs
├── Buzzhive/
│   ├── generated/
│   │   ├── fault-injection-without-skill-run1.spec.ts
│   │   ├── fault-injection-with-skill-run1.spec.ts
│   │   ├── rest-api-qa-without-skill-run1.spec.ts
│   │   ├── rest-api-qa-with-skill-run1.spec.ts
│   │   ├── claim-without-skill-run1.feature
│   │   └── claim-with-skill-run1.feature
│   └── runs/
│   └── 18 evaluation outputs
└── run-experiment.ts
```

Code: https://github.com/victor-2026/qa-automation-playwright/tree/main/experiments/skill-reliability

`Code: OrangeHRM/experiments/skill-reliability/ (local only, no GitHub remote)`

## What's Your Take? 🤔

**Have you seen skills move the needle on *novel* tasks vs. known patterns**? Share in comments — building a dataset for Part 2.

Victor Ematin · AI Quality Engineering Lead · OpenCode Go

#AIAgents #Testing #Playwright #SkillEngineering #Experiment